

ASSIGNMENT 1

Name: Gopikashree K R

Register No.: 261100690031

Subject: Linux

What If There Were No Operating System?

Question 1

What If There Were No Operating System? Imagine a computer without an operating system. Identify the problems a user would face and investigate how OS features solve them. Demonstrate at least three features using Linux and present a “**Without OS vs With OS**” comparison.

Answer

An operating system (OS) is the main software that helps a user interact with a computer and its hardware. Examples of operating systems include Linux, Windows, and mac OS.

If a computer had no operating system, using it would be very difficult. The user would have to communicate with the hardware directly. Common tasks such as creating files, running programs, managing memory, and using devices would become complicated.

Problems Without an Operating System

Without an operating system, a user would face several problems:

- There would be no simple way to run and manage applications.
- The user would have to control hardware directly.
- Managing files and folders would be difficult.
- Memory would not be automatically managed between programs.
- Multiple programs could not easily share the CPU.
- Devices such as keyboards, printers, and USB drives would need direct hardware control.
- There would be no proper interface for performing everyday computer tasks.

The operating system solves these problems by acting as a middle layer between the user, applications, and hardware.

Demonstration Using Linux

Linux provides many operating system features that make computer usage easier. Three important features demonstrated below are process management, file management, and memory management.

1. Process Management

Without OS: If there were no operating system, there would be no easy way to manage different programs running at the same time. The processor would have to be controlled directly by the programs.

With Linux: Linux manages processes and decides how much CPU time each process receives. Running processes can be viewed using the command `ps`.

The command `top` can also be used to continuously view running processes and their CPU and memory usage.

For example:

```
ps
```

```
top
```

Therefore, Linux allows multiple programs to run smoothly by managing the CPU.

2. File Management

Without OS: A user would have difficulty creating, finding, deleting, and organizing files because there would be no convenient file management system.

With Linux: Linux provides a file system and commands for managing files and directories.

For example, the following commands can be used:

```
mkdir myfolder
cd myfolder
touch example.txt
ls
```

Here, `mkdir` creates a directory, `cd` moves into the directory, `touch` creates a file, and `ls` displays the files and folders.

Thus, Linux makes storing and organizing information much easier.

3. Memory and Resource Management

Without OS: Programs would have to manage computer memory themselves. One program could interfere with memory being used by another program, which could cause errors or crashes.

With Linux: Linux manages RAM and other system resources and allocates them to running processes.

Memory usage can be checked using the command `free -h`.

For example:

```
free -h
```

This command displays information about total, used, free, and available memory.

Therefore, Linux helps programs use system resources safely and efficiently.

Without OS vs With OS

Feature	Without OS	With Linux OS
Running programs	Difficult and requires direct hardware control	Linux manages and runs processes
File management	Very difficult	Easy using files, folders, and commands
Memory management	Programs must manage memory themselves	Linux manages RAM automatically
Hardware access	Direct and complicated	Linux provides drivers and system interfaces
User interaction	Very limited	Terminal and graphical interfaces are available
CPU management	No proper scheduling	Linux schedules CPU time between processes

Conclusion

An operating system is essential for making a computer practical and easy to use. Without an OS, the user would have to deal with hardware and system resources directly, which would be complicated.

Linux solves these problems by managing processes, files, memory, hardware, and user interaction. The Linux commands demonstrated above show how an operating system performs important tasks that would otherwise be difficult for an ordinary user.

Linux Distribution comparison

Question 2

Imagine your team is setting up a cybersecurity laboratory. Explore at least five Linux distributions relevant to cybersecurity and recommend suitable distributions for different roles such as penetration testing, digital forensics, privacy, and security monitoring. Present your findings creatively as a comparison chart, poster, infographic, or “OS selection guide.”

Answer:

A cybersecurity laboratory needs different tools for different types of security work. There is no single Linux distribution that is best for every task. Some distributions are designed for penetration testing, while others are better for digital forensics, privacy, or security monitoring.

Comparison of Linux Distributions

Linux Distribution	Main Purpose	Important Features	Best Suitable Role
Kali Linux	Penetration testing	Large collection of security and network-testing tools	Penetration Testing
Parrot Security OS	Security testing and privacy	Lightweight system with security and privacy features	Penetration Testing + Privacy
Tsurugi Linux	Digital forensics	Forensic investigation and evidence-analysis tools	Digital Forensics
Security Onion	Security monitoring	Network monitoring, intrusion detection and threat hunting	Security Monitoring
Qubes OS	Privacy and isolation	Uses isolated virtual machines to separate activities	Privacy + Secure Work
BlackArch Linux	Advanced security testing	Large collection of penetration-testing tools	Advanced Penetration Testing

1. Kali Linux – Penetration Testing

Kali Linux is one of the most commonly used Linux distributions for cybersecurity learning and penetration testing.

It provides many tools for:

- Network testing
- Vulnerability assessment
- Password auditing
- Web application testing
- Wireless security testing

Recommendation: Kali Linux is a good choice for students and security professionals who want to learn penetration testing.

2. Parrot Security OS – Security and Privacy

Parrot Security OS is designed for security testing, development, and privacy. It is relatively lightweight and can be useful on computers with fewer resources.

It provides tools for:

- Penetration testing
- Digital investigation
- Network analysis
- Privacy protection

Recommendation: It is a good alternative to Kali Linux when both security tools and privacy features are required.

3. Tsurugi Linux – Digital Forensics

Tsurugi Linux is specially designed for digital forensics and incident response.

It can be used for investigating:

- Hard drives
- Deleted files
- Digital evidence
- System incidents
- Disk images

Recommendation: Tsurugi Linux is suitable for a forensic investigation team that needs to examine digital evidence.

4. Security Onion – Security Monitoring

Security Onion is focused on monitoring and defending networks. It provides tools that help security teams identify suspicious network activity.

It can be used for:

- Network monitoring
- Intrusion detection
- Threat hunting
- Log analysis
- Security investigation

Recommendation: Security Onion is a good choice for a Security Operations Center (SOC) or a laboratory focused on network security monitoring.

5. Qubes OS – Privacy and Isolation

Qubes OS uses a different approach to security. Instead of mainly focusing on penetration-testing tools, it isolates different activities using separate virtual environments.

For example, personal activities and security-related activities can be kept separated.

Recommendation: Qubes OS is suitable for privacy-conscious users and security researchers who need strong isolation between tasks.

6. BlackArch Linux – Advanced Security Testing

BlackArch Linux is based on Arch Linux and provides a large collection of cybersecurity tools.

It is useful for:

- Penetration testing
- Security research
- Vulnerability analysis
- Network security

Recommendation: BlackArch Linux is suitable for experienced users, security researchers, and professionals who are comfortable with Linux.

Recommended OS for Each Role

Cybersecurity Role	Recommended Distribution	Reason
Penetration Testing	Kali Linux	Provides many security-testing tools
Security Testing + Privacy	Parrot Security OS	Combines security tools with privacy features
Digital Forensics	Tsurugi Linux	Designed for forensic investigation
Security Monitoring	Security Onion	Designed for network monitoring and threat detection
Privacy	Qubes OS	Provides strong isolation between different activities
Advanced Security Research	BlackArch Linux	Provides a large collection of security tools

OS Selection Guide

- Need penetration testing?** Choose **Kali Linux**.

Need penetration testing with privacy? Choose **Parrot Security OS**.

Need digital evidence investigation? Choose **Tsurugi Linux**.

Need network monitoring and attack detection? Choose **Security Onion**.

Need privacy and strong isolation? Choose **Qubes OS**.

Need advanced security research? Choose **BlackArch Linux**.

Conclusion

For a cybersecurity laboratory, it is better to use different Linux distributions for different purposes rather than depending on only one.

Kali Linux can be used for penetration testing, Tsurugi Linux for digital forensics, Security Onion for security monitoring, and Qubes OS for privacy and isolation. Parrot Security OS provides a flexible alternative for security testing, while BlackArch Linux is more suitable for advanced users.

This combination gives the laboratory different environments for attacking, investigating, monitoring, and protecting systems, making it useful for practical cybersecurity learning.

Linux Commands and Exit Status

Question 3

Execute at least five Linux commands that produce both successful and unsuccessful results. Investigate their exit status using `$?`. Create a simple visual/table showing **Command** → **Result** **Exit Status** → **Meaning**. What can a script learn from an exit status?

Answer

In Linux, every command finishes with an exit status. The special variable `$?` is used to check the exit status of the most recently executed command.

Usually, an exit status of 0 means that the command was successful, while a non-zero value means that the command failed or encountered an error.

1. Successful Command: pwd

The `pwd` command displays the current working directory.

```
$ pwd
/home/student
$ echo $?
0
```

Meaning: The command executed successfully and displayed the current working directory.

2. Successful Command: mkdir

The `mkdir` command creates a new directory.

```
$ mkdir testfolder
$ echo $?
0
```

Meaning: The directory was created successfully.

3. Unsuccessful Command: ls

The `ls` command is used to display files and directories. Here, a directory that does not exist is given.

```
$ ls missingfolder
ls: cannot access 'missingfolder': No such file or directory
$ echo $?
2
```

Meaning: The command failed because the specified directory does not exist.

4. Successful Command: touch

The `touch` command can be used to create a new empty file.

```
$ touch sample.txt
$ echo $?
0
```


Meaning: The file was created successfully.

5. Unsuccessful Command: cat

The cat command displays the contents of a file. Here, a file that does not exist is used.

```
$ cat unknown.txt
cat: unknown.txt: No such file or directory
$ echo $?
1
```

Meaning: The command failed because the requested file could not be found.

6. Unsuccessful Command: cd

The cd command is used to change the current directory.

```
$ cd wrongdirectory
bash: cd: wrongdirectory: No such file or directory
$ echo $?
1
```

Meaning: The directory could not be accessed because it does not exist.

Command → Exit Status → Meaning

Command	Result	Exit Status	Meaning
pwd	Successful	0	Current directory displayed
mkdir testfolder	Successful	0	Directory created
ls missingfolder	Unsuccessful	2	Directory does not exist
touch sample.txt	Successful	0	File created
cat unknown.txt	Unsuccessful	1	File does not exist
cd wrongdirectory	Unsuccessful	1	Directory could not be accessed

Note: Exact non-zero exit codes can vary between commands and implementations. The important general rule is that 0 means success and a non-zero value means failure.

What Can a Script Learn from an Exit Status?

A script can use an exit status to understand whether a command was successful or not. Based on the status, the script can decide what to do next.

For example:

- If the command succeeds, the script can continue its execution.
- If the command fails, the script can display an error message.
- A script can perform another action when an operation fails.
- Exit status can be used for error handling and decision-making.

A simple shell example is:

```
mkdir backup
```

```
if [ $? -eq 0 ]; then
    echo "Backup directory created successfully"
else
    echo "Failed to create backup directory"
fi
```

Here, `$?` tells the script whether the `mkdir` command was successful.

Conclusion

Exit status is an important feature of Linux because it tells users and scripts whether a command completed successfully. A status of `0` indicates success, while a non-zero status indicates failure.

Scripts can use exit status for error handling, decision-making, and controlling what should happen next.